



PANTHEON CTF — Complete Solutions Guide

For Participants: This document walks you through every round of the PANTHEON CTF, explaining not just *what* to do, but *why* each step works, *how* you should think about it, and *what tools* you'd use. Treat this as a learning resource — every puzzle teaches a real-world skill.

Finding the Submission Portal

Detail	Value
Host	pantheonctf.qd.je:2222

The link to the submission portal was hidden in the `profile.jpeg` image file shared in the **WhatsApp group**.

Running `exiftool` on the image revealed:

The invitation is on. See you at `pantheonctf.qd.je` at `2222`.

The port is `2222`. Next, resolve the domain with:

```
dig TXT pantheonctf.qd.je
```

The TXT records provide the login instructions:

```
congrats! username is your pantheon team name; special characters and spaces are replaced  
by - and it is all lowercase. Password is your squad code, and the password is all  
uppercase.  
210.79.129.88 is the IP at port 2222.
```

Table of Contents

1. [Round 1 — Warmup \(The Hidden Whisper\)](#)
2. [Round 2 — Crack It If You Can \(FFT Steganography\)](#)
3. [Round 3 — The Server's Whisper \(Book Cipher & Babylonian Numbers\)](#)
4. [Round 4 — The Seeker's Files \(Babylonian PIN, MIDI Decode, Ramanujan\)](#)

5. [Round 5 — The Gatekeeper's Beacon \(XOR Known-Plaintext Attack\)](#)
 6. [Round 6 — Web: The Secure Portal \(Cookie Manipulation\)](#)
 7. [Round 7 — Web: Global Megaphone \(SSTI / Non-Recursive Blacklist\)](#)
 8. [Round 8 — Binary: Stonks Trading AI \(Format String Attack\)](#)
 9. [Round 9 — Binary: The Labyrinth \(Out-of-Bounds Memory Corruption\)](#)
 10. [Round 10 — Crypto: Sphinx of the Pantheon \(Vigenère Oracle Attack\)](#)
 11. [Rounds 11–16 — Campus Riddles](#)
 12. [Round 17 — The Unknown Location \(The Statue Waiter\)](#)
 13. [Round 18 — A Message \(Reddit & The Post from 1729\)](#)
 14. [Round 19 — The Winner \(The Dark Web Flag\)](#)
-

The Big Picture: The ARG Chain

Before diving in, understand that Rounds 1 through 4 and Rounds 17 through 19 form a **single continuous alternate-reality game (ARG) chain**. Each round's flag or output gives you the URL, password, or clue you need for the next. The remaining rounds (6–10) are another interconnected challenges in web exploitation, binary exploitation, cryptography where one flag is the key to the next and then physical campus riddles which are standalone challenges.

The chain flows like this:

```
Round 1 (intro.png hidden text)
  ↓ flag = link to image is the key
Round 2 (FFT stego on image → blog URL)
  ↓ blog URL
Round 3 (book cipher + Babylonian numbers → next site)
  ↓ Vigenère with Julius key → Round 4 URL
Round 4 (Babylonian PIN → ZIP → MIDI decode → Ramanujan → coordinates site)
  ↓ website URL
Round 17 (coordinates → find the statue → Aaron Swartz → "waiter")
  ↓ flag = r/waiters
Round 18 (Reddit r/waiters → post from 1729 → dark web link)
  ↓ .onion link
Round 19 (visit dark web link → victory flag)
```

Round 1 — Warmup

Detail	Value
Points	100
Category	Steganography / OSINT

Detail	Value
Flag	PANTHEON{https://welcome-deb.pages.dev/}
Key Tool	Browser zoom, hex editor, Morse decoder

The Setup

When the CTF begins, participants receive an image file: `intro.png`. It's the introductory poster for the event — looks like a normal event flyer. Nothing suspicious at first glance.

Step 1: Find the Visually Hidden Text

What to do: Open `intro.png` in a browser or image viewer and **zoom in**. On the image, there is **white text on a white (or near-white) background** — invisible at normal zoom levels, but if you zoom in enough or adjust contrast/brightness, the text becomes readable.

Why this works: This is one of the oldest tricks in web design and CTFs. Text can be present but invisible if its color matches the background. The human eye can't distinguish `#FFFFFF` text on a `#FEFEFE` background, but zooming in or running through an image filter reveals it.

The thinking process:

- "I have an image for Round 1. The hint says 'Seek past where the curtain falls.' That's cryptic."
- "Let me look at this image more carefully — zoom in, adjust contrast..."
- "There's hidden white text! It says something..."

Tools you could use:

- Simply zooming in your browser (`Ctrl +` or pinch-to-zoom)
- GIMP/Photoshop → Image → Adjustments → Levels (crush the whites)
- Python PIL: `ImageEnhance.Contrast(img).enhance(10.0)`

The visible flag text gives you: `PANTHEON{https://welcome-deb.pages.dev/}`

This is your Round 1 flag. Submit it.

Step 2: The Hidden Message After IEND (Bonus Discovery)

What to do: Open `intro.png` in a hex editor (HxD, xxd, or `python3 -c "open('intro.png', 'rb').read()"`). Look past the PNG `IEND` chunk marker (`49 45 4E 44 AE 42 60 82`). There is **trailing data** appended after the PNG file officially ends.

What you find: Morse code sequences and ASCII decimal values. The Morse code decodes to the flag `PANTHEON{y0u_g0t_7h15_ju11u5}`. The ASCII decimal message decodes to:

"use this as key to get through the solutions later"

Why this matters: The flag contains the word "**JUL1U5**" — a leetspeak version of "Julius". This is a reference to **Julius Caesar**, and the flag itself will be used as a **Vigenère cipher key** in Round 3. The trailing message is telling you explicitly: *save this flag, you'll need it later*.

The takeaway: In CTFs, always check image files for:

1. Visual steganography (hidden text, adjusted colors)
2. Trailing data after file markers (IEND for PNG, FFD9 for JPEG)
3. Metadata (EXIF data, comments)

Round 2 — Crack It If You Can

Detail	Value
Points	150
Category	Steganography (FFT), Web
Flag	PANTHEON{https://kryptoscicada.blogspot.com/}
Key Tool	FFT Stego CLI, browser

The Setup

After Round 1, you're directed to: <https://welcome-deb.pages.dev>

This is a **link rotator page** — it cycles through different links or content, possibly showing different things at different times or requiring interaction to find the actual content. The page also serves (or links to) an image file: [stego.jpg](#).

Step 1: Identify the Link Rotator

What to do: Visit welcome-deb.pages.dev. The page rotates links — it may redirect you or display cycling content. Inspect the page source, look at the JavaScript, and identify what's happening under the hood.

The thinking process:

- "This page seems to keep changing or redirecting me. Let me view source."
- "There's JavaScript rotating through URLs or content. Let me find the static content."
- "There's an image file — [stego.jpg](#) — linked or embedded here."

Step 2: Extract the Hidden Flag via FFT Steganography

What to do: Download `stego.jpg`. This image has a flag **hidden using Fast Fourier Transform (FFT) Block Steganography**. This is an advanced steganography technique where data is embedded in the frequency domain of image blocks rather than the spatial domain.

The thinking process:

- "Normal stego tools (steghide, zsteg, binwalk) aren't finding anything."
- "The image has a cicada image that looks like built from fourier epicycles."
- "The hint says 'FFT Block Steganography.' This is frequency-domain stego."
- "I need a specialized tool for FFT extraction."

The tool: The hint for Round 2 explicitly provides a link to the extraction tool:

```
https://gist.github.com/aj-1729/9c6a477425dbb253a0a22f645ee329ff
```

This is an FFT stego CLI tool. Run it against `stego.jpg`:

```
python3 fft_stego.py extract stego.jpg
```

What you get: The extracted data reveals the blog URL:

```
https://kryptoscicada.blogspot.com/
```

Step 3: Submit the Flag

The flag for Round 2 is the URL itself wrapped in the flag format:

```
PANTHEON{https://kryptoscicada.blogspot.com/}
```

Why FFT steganography? Traditional LSB (Least Significant Bit) steganography modifies pixel values directly and can be detected by statistical analysis. FFT steganography embeds data in the frequency components of image blocks, making it invisible to standard steganalysis tools. You need to know the specific encoding method to extract it — hence why the hint provides the exact tool.

Round 3 — The Server's Whisper

Detail	Value
Points	800
Category	Classical Cryptography, OSINT

Detail	Value
Flag	PANTHEON{raven-codes}
Key Tool	Book cipher decoder, Babylonian number converter, Vigenère cipher

The Setup

The Round 2 flag gives you the blog URL: <https://kryptoscicada.blogspot.com/>

This blog site is the challenge for Round 3. It contains two critical elements:

1. A **book cipher** encoded message
2. **Babylonian cuneiform numbers** in the background/styling

Step 1: Identify the Book Cipher

What to do: Visit the blog. You'll notice encoded text that looks like number sequences — pairs or triplets of numbers (e.g., 3:14:2 , 7:1:5). This is a **book cipher**: each number tuple references a specific word, line, or character in a known text (the "book").

The thinking process:

- "These numbers reference positions in some text. This is a book cipher."
- "But what's the book? I need the key text to decode this."
- "Where would the book come from?"

Step 2: Find the Book — The Book of the Dead

What is the book? The book for the cipher was hidden in the image `blessed.jpg` which was shared in the **WhatsApp group** as part of the CTF event communications. This image contains an **embedded ZIP file** (steganography — a ZIP appended to or hidden within the JPEG).

How to extract it:

```
# Check if there's a ZIP embedded in the image
binwalk blessed.jpg

# Extract embedded files
binwalk -e blessed.jpg

# Or manually, since ZIP files start with PK (50 4B)
# Find the ZIP offset and extract
unzip blessed.jpg # Sometimes this just works!
```

The thinking process:

- "The hint says 'The cipher is a book, you got the book right. See the book and stop the crying baby.'"

- "The 'Book of Dead' image was shared in the WhatsApp group — that's suspicious."
- "Images can contain embedded files. Let me run binwalk on it."
- "There it is — a ZIP file inside the JPEG!"
- "The extracted text is the reference book for the book cipher."

Why the Book of the Dead? It's thematic — the Pantheon CTF has mythological themes. The Egyptian Book of the Dead is a famous ancient text, fitting the "ancient knowledge" motif.

Step 3: Decode the Book Cipher

Using the extracted text as your reference book, decode the number sequences from the blog. Each number tuple points to a specific position (page/paragraph/line, line/word, character) in the extracted book text.

The decoded book cipher yields: `raven-codes.pages.dev`.

Step 4: Notice the Babylonian Numbers

What to do: Look at the blog's background, styling, or decorative elements. You'll see **Babylonian cuneiform numerals** — wedge-shaped symbols that represent numbers in base-60 (sexagesimal). These numbers become important in Round 4.

Why Babylonian numbers? This is a breadcrumb for later. The Babylonian number system uses two symbols (a vertical wedge for 1 and a corner wedge for 10) combined to form values 1-59, with positional notation for larger numbers. You'll need to understand this system for the Round 4 PIN.

Step 5: Apply the Vigenère Cipher

What to do: Take the decoded book cipher output and apply a **Vigenère cipher** using the key from Round 1.

Remember Round 1's flag: `PANTHEON{y0u_g0t_7h15_ju11u5}`

The key is **"YOUGOTTHISJULIUS"**. Julius Caesar → Caesar cipher → Vigenère cipher (the "upgraded" Caesar cipher).

The thinking process:

- "The Round 1 trailing data said 'use this as key to get through the solutions later.'"
- "I have a decoded book cipher output. The hint says 'Vigenère' and 'Julius.'"
- "Julius Caesar invented the Caesar cipher. Vigenère is its polyalphabetic evolution."
- "Let me use the Julius key from Round 1 as the Vigenère key."

```
def vigenere_decrypt(ciphertext, key):
    result = ""
    key_index = 0
    for char in ciphertext.upper():
        if char.isalpha():
            shift = ord(key[key_index % len(key)].upper()) - 65
            decrypted = chr((ord(char) - 65 - shift) % 26 + 65)
            result += decrypted
            key_index += 1
        else:
            result += char
    return result
```

What you get: The Vigenère decryption reveals the **Round 4 site URL**: `https://qrkabo1k.pages.dev/`

Submit the Flag

The Round 3 flag is the **raven-codes** obtained from the book cipher before applying Vigenère.

Round 4 — The Seeker's Files

Detail	Value
Points	2000
Category	Multi-stage puzzle (Crypto, Stego, Music, Math)
Flag	<code>PANTHEON{https://17291383220683.pages.dev/}</code>
Key Tool	Babylonian number converter, MIDI analyzer, ZIP cracker, math knowledge

This is the **big boss round** — 2000 points and the most complex multi-stage challenge in the entire CTF. It chains together Babylonian numerals, MIDI music decoding, ZIP encryption, and Ramanujan's taxicab numbers.

The Setup

From Round 3, you arrive at `https://qrkabo1k.pages.dev/` — a static site that asks for a **numeric PIN** to unlock an encrypted download.

Step 1: Enter the Babylonian PIN

What to do: The site has a PIN entry field and a **sun icon** as a decorative element which was in round 3 in the image containing babylonian numbers. The PIN is written in **Babylonian cuneiform numerals** — the same number system you saw in the blog background in Round 3 read it from the top left corner in **anti-clockwise** order.

The PIN: 20092002194507165301101201319690815

How to derive it: The Babylonian numbers on the blog and/or the Round 4 site translate to this decimal sequence. The **sun icon** is the critical hint — it tells you the **reading direction**. In Babylonian writing, the sun's position indicates which direction to read the cuneiform wedges (left-to-right or right-to-left).

The thinking process:

- "I see wedge-shaped symbols. These are Babylonian cuneiform numbers."
- "There's a sun icon. Why? In ancient Mesopotamian writing, the sun indicated reading direction."
- "The sun tells me to read the numbers in a specific direction (e.g., left to right)."
- "Converting each group of wedges: vertical wedges = 1s, corner wedges = 10s..."
- "The full number is: 20092002194507165301101201319690815 "

Technical details (from the build script):

```
PIN = "20092002194507165301101201319690815"
```

The site uses AES-256-GCM encryption with this PIN as the key derivation input (PBKDF2 with 100,000 iterations). When you enter the correct PIN, it decrypts the payload and initiates a ZIP file download.

Step 2: Open the Outer ZIP

What you get: round4_challenge.zip — an unencrypted ZIP containing exactly two files:

1. ancient_hymn.mid — A MIDI music file
2. secret_archive.zip — A password-protected ZIP (ZipCrypto encrypted)

Step 3: Decode the MIDI File

What to do: Open ancient_hymn.mid in a MIDI editor/viewer (like MuseScore, MIDI Explorer, or a Python MIDI library). The file has two tracks:

- **Track 1: "Melody - The Oracle's Voice"** — This is the important one
- **Track 2: "Harmony - Babylonian Lyre"** — Decorative accompaniment (red herring)

The key insight: Track 1 maps **each letter of the alphabet to a MIDI pitch**:

- A = pitch 60 (Middle C)
- B = pitch 61
- C = pitch 62
- ...
- Z = pitch 85

How to decode:

```
import mido # or any MIDI library

mid = mido.MidiFile('ancient_hymn.mid')
password = ""
for track in mid.tracks:
    if "Melody" in track.name:
        for msg in track:
            if msg.type == 'note_on' and msg.velocity > 0:
                letter = chr(msg.note - 60 + ord('A'))
                password += letter
print(password) # → BABYLONIANHARMONY
```

The thinking process:

- "A MIDI file in a CTF? The notes must encode something."
- "The hint says '26 pitches map directly to the alphabet.' That's A-Z = 26 letters."
- "If A=60 (Middle C), then each subsequent pitch is the next letter."
- "Reading the melody notes: 61, 60, 61, 88... wait, let me map these..."
- "B-A-B-Y-L-O-N-I-A-N-H-A-R-M-O-N-Y → BABYLONIANHARMONY!"

The password is: **BABYLONIANHARMONY**

Step 4: Unlock the Inner ZIP

What to do: Use the decoded MIDI password to open `secret_archive.zip`:

```
unzip -P "BABYLONIANHARMONY" secret_archive.zip
```

What you get: `flag.txt` — but it's NOT a simple flag. It contains:

1. **ASCII art of Srinivasa Ramanujan** (the famous mathematician)
2. A **cryptic riddle** about "three brothers" who "share one secret"

Step 5: Understand the Ramanujan Riddle

The riddle in `flag.txt` says:

"I am the third of my kind, yet I arrived first to your eye. My brothers did not ride with me — they hid in plain sight..."

"One dressed himself in pixels, wearing width like a crown and height like a throne."

"The other slept inside a silence older than your calendar, a passenger in the bones of light itself..."

"We are three. We share one secret — every one of us can be broken into two pairs of cubes, and every one of us knows the same address."

"When all three stand together, separated by nothing but a dot, append our kingdom and knock."

The key concept: Taxicab Numbers (Ramanujan Numbers)

A **taxicab number** (or Hardy-Ramanujan number) is a number that can be expressed as the sum of two cubes in two different ways. The most famous is:

$$1729 = 1^3 + 12^3 = 9^3 + 10^3$$

This was famously identified by Ramanujan when G.H. Hardy mentioned arriving in taxi number 1729, calling it "dull." Ramanujan instantly replied it was actually very interesting.

The riddle speaks of **three** such numbers. The three Ramanujan/taxicab numbers are:

1. **1729** — The classic (Ta(2))
2. **The image dimensions** — `intro.png`'s width and height ARE Ramanujan numbers
3. **The third** — found from the ASCII art file itself (the value 1729 appears as the "third of my kind")

The thinking process:

- "ASCII art of Ramanujan + a riddle about numbers that split into pairs of cubes."
- " $1729 = 1^3 + 12^3 = 9^3 + 10^3$. That's the Hardy-Ramanujan number."
- "'One dressed himself in pixels, wearing width like a crown...' — that's describing an IMAGE."
- "The `intro.png` from Round 1! Let me check its dimensions."
- "The riddle says 'three brothers.' I need three taxicab numbers."

Checking `intro.png` dimensions:

```
identify intro.png # ImageMagick
# Or
python3 -c "from PIL import Image; print(Image.open('intro.png').size)"
```

The dimensions of `intro.png` are Ramanujan numbers! The three numbers are:

- **1729** (from the riddle/ASCII art)
- **13832** (one dimension — this is a taxicab number: can be expressed as sum of two cubes in two ways)
- **20683** (another dimension/derivation)

Step 6: Construct the URL

What to do: Concatenate the three Ramanujan numbers and append `.pages.dev`:

```
1729 + 13832 + 20683 = "17291383220683"
→ https://17291383220683.pages.dev/
```

The flag: `PANTHEON{https://17291383220683.pages.dev/}`

This URL is the gateway to Round 17 — the challenge continues from here as a physical/OSINT trail.

Round 5 - URL hint

The Setup

Welcome Seekers, a soul overhead goddess Athena saying something

Thy valour, mortal, doth Athena please, Thus grant I thee a path across the seas. To forge the cipher where thy secrets lie, Bind every token as declare do I: Begin with h-t-t-p-s, secure and true, A colon and twain forward slashes too (://). Then write the fleet-foot herald, hermes hight, And strike a hyphen (-) to divide the night. Inscribe the token beacon, burning bright, And strike a second hyphen (-) in thy sight. Then pen the cipher latest, newly born, And drop a single speck (.) to greet the morn. Set down onrender, where great visions wake, And cast a final speck (.) for valour's sake. Seal thou with com to end this mystic spell, And seek the hidden shore where champions dwell!

Lets see if its for your soul or not ! -1729

This gives the link to the round 5 url `https://hermes-beacon-latest.onrender.com`

Round 5 — The Gatekeeper's Beacon

Detail	Value
Points	300
Category	Cryptography (XOR)
Flag	<code>PANTHEON{g4t3w4y_unl0ck3d_https://web-secure-portal.onrender.com}</code>
Key Tool	Python, XOR known-plaintext attack

The Setup

Round 5 is accessed at `https://hermes-beacon-latest.onrender.com` — a terminal-style web interface themed as a "Pantheon Defense Network" intercepted transmission. It uses **XOR-8 signal modulation** and asks for an 8-character "divine access key."

Step 1: Understand the Encryption

The challenge gives you:

- An encrypted payload (array of hex bytes)
- The modulation type: **XOR-8** (8-byte repeating key XOR cipher)

- You need the 8-character key to decrypt

```
_PAYLOAD = [  
    0x18, 0x04, 0x1c, 0x19, 0x0d, 0x16, 0x7d, 0x78,  
    0x33, 0x22, 0x66, 0x39, 0x76, 0x24, 0x06, 0x4f,  
    # ... (65 bytes total)  
]
```

Step 2: Known-Plaintext Attack

The key insight: You KNOW the decrypted flag starts with `PANTHEON{`. That's 9 characters, and the key is only 8 characters. This means you can recover the **entire key** through a known-plaintext attack.

How XOR works: `ciphertext[i] = plaintext[i] XOR key[i % 8]`

Therefore: `key[i % 8] = ciphertext[i] XOR plaintext[i]`

```
payload = [0x18, 0x04, 0x1c, 0x19, 0x0d, 0x16, 0x7d, 0x78, ...]  
known_header = "PANTHEON{"  
  
# Recover the 8-byte key  
key = ""  
for i in range(8):  
    key += chr(payload[i] ^ ord(known_header[i]))  
print(f"Recovered Key: {key}")  
# → HERMES26
```

The thinking process:

- "XOR cipher with 8-byte key. The decrypted text starts with 'PANTHEON{' — that's the standard flag format."
- "If I XOR the known plaintext against the ciphertext, I get the key!"
- "P XOR 0x18 = H, A XOR 0x04 = E, N XOR 0x1C = R, T XOR 0x19 = M..."
- "The key is HERMES26!"

Step 3: Decrypt the Full Message

```
key = "HERMES26"  
flag = ""  
for i, byte in enumerate(payload):  
    flag += chr(byte ^ ord(key[i % len(key)]))  
print(flag)  
# → PANTHEON{g4t3w4y_unl0ck3d_https://web-secure-portal.onrender.com}
```

Submit: `PANTHEON{g4t3w4y_unl0ck3d_https://web-secure-portal.onrender.com}`

The lesson: XOR encryption is only as strong as the key's secrecy. If an attacker knows ANY portion of the plaintext equal to the key length, they can recover the entire key. This is why XOR is never used alone in real cryptography — it's always combined with other techniques (AES, ChaCha20, etc.).

Round 6 — Web: The Secure Portal

Detail	Value
Points	350
Category	Web Exploitation
Flag	PANTHEON{c00k13_m4n1pul4t10n_https://web-global-megaphone.onrender.com}
Key Tool	Browser DevTools, base64 decoder

The Setup

Visit <https://web-secure-portal.onrender.com>. You see a login page with ominous "quantum encryption" warnings. There's also a registration page.

Step 1: Recognize the Trap

The entire login page is a decoy. The server's login handler is hardcoded to ALWAYS fail, no matter what credentials you enter. It displays scary error messages about "quantum integrity mismatch" and "account locked" — all fake.

The code literally does this:

```
// ...but we just hardcode a failure no matter what they do!
errorMsg := "ERR_0x44: Quantum integrity mismatch. Account locked..."
```

There's also a fake SQL query string designed to make you waste time attempting SQL injection:

```
_ = fmt.Sprintf("SELECT * FROM users WHERE username = '%s' AND password = '%s'",
username, hash)
```

The thinking process:

- "Login always fails. SQL injection doesn't work. Quantum errors? That seems fake."
- "The hint says 'Check the auth_token cookie issued after registration.'"
- "Let me register an account instead and look at the cookie."

Step 2: Register and Inspect the Cookie

What to do:

1. Go to `/register` and create any account (username: `test` , password: `anything`)
2. Open Browser DevTools → Application → Cookies
3. Find the `auth_token` cookie

The cookie value looks like: `dGVzdDp1c2Vy`

That trailing `=` is a dead giveaway for **Base64 encoding**.

Step 3: Decode and Modify the Cookie

```
echo "dGVzdDp1c2Vy" | base64 -d
# → test:user
```

The cookie format is `username:role` — Base64 encoded. Your role is `user` .

Change it to admin:

```
echo -n "test:admin" | base64
# → dGVzdDphZG1pbG==
```

Step 4: Replace the Cookie and Visit Dashboard

1. In DevTools → Cookies, replace the `auth_token` value with `dGVzdDphZG1pbG==`
2. Navigate to `/` (the dashboard)
3. The server checks `role == "admin"` and reveals the flag!

Flag: `PANTHEON{c00k13_m4n1pul4t10n_https://web-global-megaphone.onrender.com}`

The lesson: Never trust client-side authentication tokens. This challenge demonstrates why:

- Cookies should be **signed** (HMAC) or **encrypted** so clients can't tamper with them
- Role information should come from the server's session store, not the cookie itself
- Fake "quantum" security theater doesn't replace real access controls

Round 7 — Web: Global Megaphone

Detail	Value
Points	400

Detail	Value
Category	Server-Side Template Injection (SSTI)
Flag	PANTHEON{ssti_g0_t3mpl4t3_https://pwn-stonks-fmt.onrender.com}
Key Tool	Browser, Go template syntax knowledge

The Setup

Visit `https://web-global-megaphone.onrender.com`. It's an "announcement board" where you type a message and it gets displayed. The developer brags about input sanitization.

Step 1: Identify SSTI

What to do: Try entering Go template syntax: `{{ . }}`

The server uses Go's `html/template` engine and **compiles your input as a template**, then executes it against a struct containing the flag. This is **Server-Side Template Injection (SSTI)**.

The thinking process:

- "The site says 'I heard templating is a cool and modular way to build web apps!'"
- "If my input is being processed as a template, I can inject template directives."
- "In Go templates, `{{ .Flag }}` would access a `Flag` field on the data struct."

Step 2: Bypass the Sanitization

If you try `{{ .Flag }}`, it won't work! The server strips the word `Flag` from your input:

```
sanitizedInput := strings.ReplaceAll(userInput, "Flag", "")
```

But this replacement is non-recursive — it only runs once. If you nest `Flag` inside itself, the outer shell survives after the inner one is removed.

Step 3: Craft the Nested Payload

```
{{ .FlFlagag }}
```

Here's what happens:

1. Your input: `{{ .FlFlagag }}`
2. Server removes `Flag`: `{{ .Fl____ag }}` → `{{ .Flag }}`
3. The template engine executes `{{ .Flag }}` against `SecretData{Flag: "PANTHEON{...}"}`
4. **The flag is rendered!**

Submit: `PANTHEON{ssti_g0_t3mpl4t3_https://pwn-stonks-fmt.onrender.com}`

The lesson: Non-recursive blacklist sanitization is fundamentally broken. An attacker can always construct a payload where removing the blacklisted word produces the blacklisted word. Proper defenses include:

- **Recursive sanitization** (loop until no more matches)
- **Allowlisting** instead of blacklisting
- **Parameterized templates** (never compile user input as template code)
- **Sandboxed template engines** with restricted functionality

Round 8 — Binary: Stonks Trading AI

Detail	Value
Points	450
Category	Binary Exploitation (Format String)
Flag	<code>PANTHEON{f0rm4t_5tr1ng_https://pwn-maze-oob-1.onrender.com}</code> (Base64 encoded on stack as <code>UEF0VEhFT057ZjBybTR0XzV0cjFuZ19tNHN0M3J9</code>)
Key Tool	Terminal/netcat, Base64 decoder

The Setup

Connect to `https://pwn-stonks-fmt.onrender.com` — a terminal-based "stock trading" application. It asks for an "API token" to authorize trades.

Step 1: Identify the Format String Vulnerability

The program reads your input and passes it directly to `printf()` without a format specifier:

```
scanf("%299s", api_token);  
printf(api_token); // VULNERABILITY: should be printf("%s", api_token)
```

This means your input is interpreted as a **format string**. If you type `%p`, `printf` treats it as a pointer format specifier and reads values from the stack.

The thinking process:

- "It asks for an API token and then 'processes' it. Let me try something unusual."
- "What if I enter `%p` ? If the input is used as a format string..."

- "It printed a hex address! That's a format string vulnerability!"

Step 2: Leak the Stack

The secret flag is stored as a Base64-encoded local variable on the stack:

```
char secret_b64[] = "UEF0VEhFT057ZjBybTR0XzV0cjFuZ19tNHN0M3J9";
```

Since it's a local array, it lives on the stack right near the `api_token` buffer. By chaining multiple `%p` specifiers, you can read sequential stack values:

```
API Token: %p.%p.%p.%p.%p.%p.%p.%p.%p.%p.%p.%p.%p.%p
```

This dumps hex values from the stack. Some of these values, when converted to ASCII, reveal the Base64 string.

Alternatively, use `%s` to read strings directly from the stack:

```
API Token: %s
```

Or use positional arguments: `%7$s` to read the 7th argument on the stack as a string.

Step 3: Decode the Base64

```
echo "UEF0VEhFT057ZjBybTR0XzV0cjFuZ19tNHN0M3J9" | base64 -d  
# → PANTHEON{f0rm4t_5tr1ng_https://pwn-maze-oob-1.onrender.com}
```

Flag: `PANTHEON{f0rm4t_5tr1ng_https://pwn-maze-oob-1.onrender.com}`

The lesson: Format string vulnerabilities are among the most dangerous in C/C++. They allow attackers to:

- **Read arbitrary memory** (`%p` , `%x` , `%s`)
- **Write arbitrary memory** (`%n`)
- **Control program execution flow**

Always use `printf("%s", user_input)` instead of `printf(user_input)`.

Round 9 — Binary: The Labyrinth

Detail	Value
Points	500

Detail	Value
Category	Binary Exploitation (Out-of-Bounds Memory)
Flag	PANTHEON{0ut_0f_b0unds_https://crypto-sphinx-oracle.onrender.com}
Key Tool	Terminal/netcat, understanding of memory layout

The Setup

Connect to `https://pwn-maze-oob-1.onrender.com` — a text-based maze game. You control a player (`@`) on a 30×90 grid and need to reach the exit (`x`) at position (29, 89).

Step 1: Reach the Exit (and Fail)

If you navigate to the exit normally, you get:

```
"You reached the exit, but you don't have the secret key!"
```

The win condition checks `state.win_flag != 0`, but `win_flag` is initialized to 0. You can't set it through normal gameplay.

Step 2: Identify the Out-of-Bounds Vulnerability

The movement controls have **NO bounds checking**:

```
if(move == 'w') player_x--;      // Can go negative!  
else if(move == 's') player_x++; // Can exceed 29!  
else if(move == 'a') player_y--; // Can go negative!  
else if(move == 'd') player_y++; // Can exceed 89!
```

And there's a "breadcrumb" command (`p`) that writes to the map at the player's position:

```
else if(move == 'p') {  
    state.map[player_x][player_y] = 'P'; // Writes to arbitrary memory!  
}
```

Step 3: Understand the Memory Layout

The `GameState` struct is:

```
struct GameState {  
    char map[30][90]; // 2700 bytes  
    int win_flag;      // 4 bytes, immediately after map  
};
```

`win_flag` sits at byte offset 2700 from the start of `map`. Since `map[i][j]` accesses byte `i*90 + j`, we need `i*90 + j = 2700`.

Calculate the position: `2700 / 90 = 30`, remainder `0`. So `map[30][0]` is the first byte of `win_flag`.

Step 4: Exploit

1. Move the player to position (30, 0) — that's one step past the bottom edge of the map
2. Press `p` to drop a breadcrumb — this writes `'P'` (ASCII 80) to `win_flag`
3. Navigate back to the exit at (29, 89)
4. The `win_flag` is now non-zero, so the win condition triggers!

```
# Move to row 30 (starting from row 11):
# Press 's' 19 times to reach row 30
# Move to column 0:
# Press 'a' 46 times to reach column 0
# Drop breadcrumb:
# Press 'p'
# Navigate to exit (29, 89):
# Press 'w' once (to row 29)
# Press 'd' 89 times (to column 89)
```

Flag: `PANTHEON{Out_of_b0unds_https://crypto-sphinx-oracle.onrender.com}`

The lesson: Bounds checking is not optional. The lack of array bounds validation in C/C++ is the root cause of countless real-world vulnerabilities (buffer overflows, heap overflows, etc.). In this case, writing one byte past the array boundary corrupted an adjacent variable — in real software, this could overwrite return addresses, function pointers, or security-critical flags.

Round 10 — Crypto: Sphinx of the Pantheon

Detail	Value
Points	600
Category	Cryptanalysis (Vigenère)
Flag	<code>PANTHEON{Cr4ck1ng_Th3_0lymp14n_0r4cl3_c0mpl3t3d}</code>
Key Tool	Python, understanding of Vigenère cipher

The Setup

Connect to `https://crypto-sphinx-oracle.onrender.com` — an interactive Sphinx that:

1. Shows you an encrypted offering: `OLQNXMFRRGEISZ` (the Vigenère encryption of `OLYMPIANNECTAR` with key `ZEUS`)
2. Lets you encrypt your own plaintext (oracle access) — but only **4 times**
3. Asks you to guess the original plaintext offering

Step 1: Understand the Vigenère Cipher

The Vigenère cipher encrypts each letter by shifting it by the corresponding key letter's position:

Plaintext:	O	L	Y	M	P	I	A	N	N	E	C	T	A	R
Key:	Z	E	U	S	Z	E	U	S	Z	E	U	S	Z	E
Shift:	25	4	20	18	25	4	20	18	25	4	20	18	25	4
Ciphertext:	N	P	S	E	O	M	U	F	M	I	W	L	Z	V

(The exact ciphertext depends on the implementation.)

Step 2: Recover the Key Using the Oracle

You have 4 encrypt queries. Use them strategically:

Query 1: Encrypt `AAAA` → The output directly reveals the first 4 key letters (since shifting 'A' by K gives K).

- If `AAAA` → `ZEUS`, the key starts with `ZEUS`.

Query 2: Encrypt `AAAAAAAAAA` (10 A's) → Reveals the full repeating key pattern.

- Output: `ZEUSZEUSZE` → Key is `ZEUS` (4-letter repeating key)

Now you know the key is `ZEUS`.

Step 3: Decrypt the Offering

```
key = "ZEUS"
ciphertext = "OLQNXMFRRGEISZ" # The encrypted offering shown by the Sphinx

plaintext = ""
key_index = 0
for char in ciphertext:
    if char.isalpha():
        shift = ord(key[key_index % len(key)]) - 65
        decrypted = chr((ord(char) - 65 - shift) % 26 + 65)
        plaintext += decrypted
        key_index += 1
    else:
        plaintext += char

print(plaintext) # → OLYMPIANNECTAR
```

Step 4: Submit the Guess

Enter `OLYMPIANNECTAR` when the Sphinx asks for the offering.

Flag: `PANTHEON{Cr4ck1ng_Th3_0lymp14n_0r4cl3_c0mp13t3d}`

The lesson: The Vigenère cipher was considered "unbreakable" for 300 years, but it's trivially broken when:

1. You have **oracle access** (chosen-plaintext attack) — as in this challenge
2. You know the **key length** (use Kasiski examination or index of coincidence)
3. You have enough **ciphertext** (frequency analysis on each key position)

Modern ciphers (AES, ChaCha20) are designed to resist all of these attacks.

Rounds 11–16 — Campus Riddles

These six rounds are **physical/OSINT challenges** where participants must solve riddles to identify specific locations on the BIT Mesra campus, then find QR codes or flags at those locations. Each is worth **100 points** and limited to **3 solves** (first 3 teams only).

Round 11 — Campus Riddle 1: Blood Donation Camp / Management Dept Junction

Riddle:

*"I face the house where managers are made, standing where a bent road forks in the shade.
Once a year, veins offer up their red..."*

Solution logic:

- "House where managers are made" → **Management Department**
- "Veins offer up their red" → **Blood donation camp** (annual event)
- "Bent road forks" → Road junction/fork near the dept
- "Third-years rest in twin numbers" → Hostel with double-digit number (e.g., Hostel 11)
- "Official wheels roll through me" → Campus vehicle route from the yard to the exit

Location: The junction near the Management Department where the blood donation camp is held (Dispensary).

Flag: Found at the physical location (QR code or written flag). The flag hash in the system is `de3d67f...` — participants must visit the location.

Round 12 — Campus Riddle 2: The Red Food Truck

Riddle:

"Among many wheels that serve the hungry crowd, one glows in a hue that warns you to stop... Right across the road, first-years find their bed..."

Solution logic:

- "Wheels that serve the hungry crowd" → **Food carts/stalls**
- "Hue that warns you to stop" → **Red** (stop sign color)
- "Fresh fruits and cold juices" → **Juice cart**
- "Right across the road, first-years find their bed" → **Opposite Hostel 1** (first-year hostel)

Location: The red food truck opposite Hostel 4.

Flag: `PANTHEON{m1x3d_v3g_1s_g04t3d}` (found in the `ads.txt` file of the associated website: `website2`).

How to find it digitally: The campus riddle's associated website has an `ads.txt` file at the root. Checking `/ads.txt` (a standard web file for ad verification) reveals the flag hidden among fake ad records.

Round 13 — Campus Riddle 3: The Gym / PT Department

Riddle:

"Not the gate, but near enough to see, where sweat replaces a library's decree. Among five paths a student may pick, one's a joke, a lazy trick..."

Solution logic:

- "Sweat replaces a library's decree" → **Gym/Physical Training** (exercise vs. studying)
- "Five paths a student may pick, one's a joke" → Five academic departments, PT/Physical Education is the "lazy" option
- "One voice commands here, strict yet sly" → **PT instructor**
- "A younger soul lifts spirits" → **Gym assistant/trainer**

Location: The Gym / PT Department area.

Flag: `PANTHEON{I3T3_1s_n0t_4_t3ch_club}` (hidden in an HTML comment at line 283 of `website1/index.html`, buried among hundreds of fake "system log" comments).

How to find it digitally: View the page source of the associated website. Scroll through the HTML comments (which look like system logs with hundreds of fake entries like `[125] document registry`

nominal). At line 283, hidden among them:

```
[129] PANTHEON{I3T3_1s_n0t_4_t3ch_club}
```

Round 14 — Campus Riddle 4: Tea Stall near Student Activity Centre

Riddle:

"Near the ring where echoes gather round, steam rises where chatter is always found. Once I offered three ways to pay, a middle path has walked away..."

Solution logic:

- "Ring where echoes gather" → **Student Activity Centre** (amphitheatre/auditorium)
- "Steam rises, chatter" → **Tea stall**
- "Three ways to pay, middle path walked away" → Used to accept 3 denominations, now only smallest and largest
- "Campus's favorite brew" → **Tea** (chai)

Location: The tea stall near the Student Activity Centre.

Flag: Found at the physical location.

Round 15 — Campus Riddle 5: DOSA (Dean of Student Affairs) Office

Riddle:

"A home it once was, walls that used to rest, now it logs your leave, your ticket request. The first door you open when you first arrive..."

Solution logic:

- "Logs your leave" → **Administrative office** handling leave requests
- "First door when you first arrive" → **Student Affairs** (first stop for new students)
- "Gateway to grades, digital hive" → **Academic records/registration**
- "Complaints and requests flow through" → **Dean of Student Affairs (DOSA)**
- "Eyes gaze from far and wide, reading the earth" → **Remote Sensing Department** (nearby)

Location: The DOSA (Dean of Student Affairs) office, near the Remote Sensing Department.

Flag: Found at the physical location.

Round 16 — Campus Riddle 6: Bit Soft Kiosk

Riddle:

"Tucked away, flanked by two silent giants, a tiny market hides in quiet defiance. Our college's name ends in a word so grand, strip away its final syllable's strand..."

Solution logic:

- "Two silent giants" → **Two large hostels** (Hostels 6 and 7)
- "Our college's name ends in a word so grand" → BIT **Mesra** → "Mesra"
- "Strip away its final syllable" → Remove "ra" → "Mes" → no... Think differently: BIT = Birla Institute of Technology → "Technology" → strip final syllable "gy" → "Technolo"... Actually: the college is "BIT" → the word is "Bit" → strip the ending → "Bi"
- "Cold bottles fizz with a word that's tame, not hard, not sharp, but gentle" → **"Soft"** (as in soft drinks)
- "Bit" + "Soft" = **"BitSoft"** → **Bit Soft kiosk**

Location: The Bit Soft kiosk/shop between Hostels 6 and 7 (Techno).

Flag: Found at the physical location.

Round 17 — The Unknown Location

Detail	Value
Points	200
Category	OSINT / Lateral Thinking
Flag	PANTHEON{r/waiters}
Key Tool	Google Maps, Wikipedia, lateral thinking

The Setup

From Round 4, you reached <https://17291383220683.pages.dev/> . This page displays:

37°46'56.3"N 122°28'17.65"W

And hidden in tiny black-on-black text at the bottom:

find the statue waiter

Step 1: Identify the Coordinates

What to do: Plug the coordinates into Google Maps:

37°46'56.3"N 122°28'17.65"W

This points to a location in **San Francisco**, near the Internet Archive headquarters.

Step 2: Find the Statue

The thinking process:

- "The hidden hint says 'find the statue waiter.' There's a statue at these coordinates."
- "Searching for statues near the Internet Archive in San Francisco..."
- "There's a statue/memorial of **Aaron Swartz** near there!"

Who is Aaron Swartz? Aaron Swartz (1986–2013) was a programming prodigy, internet activist, and co-founder of Reddit. He was instrumental in creating RSS, Creative Commons, and the Open Library. He was also a key figure in the fight against SOPA/PIPA. His tragic story is deeply connected to internet freedom and open access.

Step 3: Connect "Statue" + "Waiter"

The key lateral thinking leap:

The hint says "find the statue **waiter**." This is a play on words:

- Aaron Swartz → "Swartz" sounds like... nothing directly.
- But the connection is: Aaron Swartz co-founded **Reddit**.
- A "waiter" is someone who serves/waits tables.
- The plural of "waiter" is "**waiters**".
- On Reddit, communities are called **subreddits** (r/something).

The flag is: PANTHEON{r/waiters}

The thinking process:

- "Statue = Aaron Swartz. He co-founded Reddit."
- "The hint says 'waiter.' Waiters... serve... tables..."
- "r/waiters is a real subreddit where restaurant servers vent about work!"
- "The hint from rounds.yaml says: 'the ones taking the orders, not cooking the food' — that's waiters!"
- "The 7-letter plural noun for their job title = 'waiters'"

Submit: PANTHEON{r/waiters}

Round 18 — A Message

Detail	Value
Points	300
Category	OSINT / Reddit
Flag	(The dark web .onion URL wrapped in PANTHEON{})
Key Tool	Reddit, careful reading

The Setup

From Round 17, you know to go to **Reddit's r/waiters** subreddit. Round 18 is titled "a message" and the hint says:

"The thread has a message somewhere in it, but it's not in plain sight. Find it."

Step 1: Search r/waiters for "1729"

What to do: Go to `reddit.com/r/waiters` and search for posts related to **1729** — the Ramanujan number that's been a recurring theme throughout the CTF.

The thinking process:

- "Round 18 is about finding 'a message.' The subreddit is r/waiters."
- "1729 keeps coming up — Ramanujan's taxicab number, the website URL, the riddle..."
- "Let me search for '1729' in r/waiters."

Step 2: Find the Post

You find a post titled something like **"a message from 1729"** (or containing 1729 in its content). The post appears to be a normal waiter story/vent, but hidden within it — perhaps in:

- A hyperlink disguised as normal text
- Encoded text in the post body
- A comment buried in the thread
- Unicode steganography or zero-width characters

Step 3: Extract the Dark Web Link

The post contains a **.onion link** (a Tor hidden service URL). This is the gateway to Round 19.

The thinking process:

- "The hint says the message is 'not in plain sight.' I need to look more carefully."
- "Maybe it's in the HTML source, or hidden with formatting tricks."
- "Found it — there's a .onion link hidden in the post!"

Submit: The .onion URL (or a derivative) wrapped in the PANTHEON{} flag format.

Round 19 — The Winner

Detail	Value
Points	700
Category	OSINT / Dark Web
Flag	(Displayed on the .onion site)
Key Tool	Tor Browser
Max Solves	3 (first 3 teams only!)

The Setup

From Round 18, you have a .onion URL — a dark web address only accessible through the **Tor network**.

Step 1: Access Via Tor

What to do:

1. Download and install **Tor Browser** (<https://www.torproject.org/>)
2. Open the .onion link in Tor Browser
3. The page loads and displays the **final victory flag**

Step 2: Claim Victory

The page shows the flag directly. Submit it to the portal.

This round has a max of 3 solves — only the first 3 teams to reach this point and submit the flag earn the 700 points. This makes the entire ARG chain a race.

The lesson: The Tor network provides anonymous, censorship-resistant communication. .onion addresses use cryptographic keys as their URL, making them self-authenticating. While often associated with illicit activity, Tor is a critical tool for:

- Journalists in authoritarian regimes
- Whistleblowers (SecureDrop)
- Privacy-conscious individuals

- Circumventing censorship
-

Appendix: Tools & Resources

Recommended CTF Toolkit

Category	Tools
Steganography	<code>binwalk</code> , <code>steghide</code> , <code>zsteg</code> , <code>stegsolve</code> , FFT stego tools, <code>exiftool</code>
Hex Editing	HxD (Windows), <code>xxd</code> (Linux), <code>hexdump</code>
Cryptography	CyberChef, <code>hashcat</code> , Python <code>cryptography</code> library, dCode.fr
Web	Browser DevTools (F12), Burp Suite, <code>curl</code> , Postman
Binary	GDB, <code>pwntools</code> , Ghidra, IDA Free, <code>ltrace</code> / <code>strace</code>
OSINT	Google Maps, Google Dorking, Wayback Machine, <code>sherlock</code>
Networking	Wireshark, <code>netcat</code> / <code>ncat</code> , Tor Browser
Music/Audio	MuseScore, Audacity, Python <code>mido</code> / <code>music21</code>
Image	GIMP, Python PIL/Pillow, ImageMagick

Key Concepts Tested

1. **Steganography** — Hiding data in images (visual, FFT, trailing data, embedded files)
2. **Classical Cryptography** — Caesar, Vigenère, book ciphers, XOR
3. **Web Security** — Cookie tampering, SSTI, format strings, base64 encoding
4. **Binary Exploitation** — Format string attacks, out-of-bounds memory access
5. **OSINT** — Coordinate lookup, historical figure identification, social media investigation
6. **Number Theory** — Babylonian numerals, Ramanujan taxicab numbers
7. **Music Theory** — MIDI encoding, pitch-to-letter mapping
8. **Dark Web** — Tor, .onion services
9. **Lateral Thinking** — Word play, cross-referencing clues across rounds

The Full Flag Chain (Rounds 1 → 4 → 17 → 19)

R1: PANTHEON{https://welcome-deb.pages.dev/} (hidden white text on intro.png)
R2: PANTHEON{https://kryptoscicada.blogspot.com/} (FFT stego extraction)
R3: PANTHEON{https://raven-codes.pages.dev/} (book cipher + Vigenère)
R4: PANTHEON{https://17291383220683.pages.dev/} (MIDI + Ramanujan numbers)
R17: PANTHEON{r/waiters} (coordinate OSINT → Aaron Swartz)
R18: (dark web link) (Reddit post from 1729)
R19: (victory flag) (Tor hidden service)

The Second flag chain (Rounds 5 → 6 → 7 → 8 → 9 → 10)

R5: PANTHEON{g4t3w4y_unl0ck3d_https://web-secure-portal.onrender.com} (XOR known-plaintext)
R6: PANTHEON{c00k13_m4n1pul4t10n_https://web-global-megaphone.onrender.com} (cookie manipulation)
R7: PANTHEON{ssti_g0_t3mpl4t3_https://pwn-stonks-fmt.onrender.com} (SSTI bypass)
R8: PANTHEON{f0rm4t_5tr1ng_https://pwn-maze-oob-1.onrender.com} (format string)
R9: PANTHEON{0ut_of_b0unds_https://crypto-sphinx-oracle.onrender.com} (OOB write)
R10: PANTHEON{Cr4ck1ng_Th3_0lymp14n_0r4cl3_c0mpl3t3d} (Vigenère oracle)

Standalone Round Flags

R11: PANTHEON{tH1S_1S_@_F@Ls3_FL@G}
R12: PANTHEON{m1x3d_v3g_1s_g04t3d}
R13: PANTHEON{I3T3_1s_n0t_4_t3ch_club}
R14: PANTHEON{P@g@l_T0_B@n@_DUnG@_D3ZT_N0}
R15: PANTHEON{wh@7_c0l0ur_!\$_y0ur_c9}
R16: PANTHEON{!E7_0r_IEEE_wh!ch_club_w! Il_y0u_ch00se}